IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 8 September 2024, accepted 23 September 2024, date of publication 27 September 2024, date of current version 7 October 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3469636

RESEARCH ARTICLE

Siamese Neural Networks Method for Semantic Requirements Similarity Detection

NOJOOM A. ALNAJEM, (Member, IEEE), MANAL BINKHONAIN, AND M. SHAMIM HOSSAIN, (Senior Member, IEEE)

Department of Software Engineering, College of Computer and Information Sciences, King Saud University, Riyadh 12372, Saudi Arabia

Corresponding authors: Manal Binkhonain (mbinkhonain@ksu.edu.sa) and M. Shamim Hossain (mshossain@ksu.edu.sa)

This work was supported by King Saud University, Riyadh, Saudi Arabia, through the Researchers Supporting Project number (RSP2024R32).

ABSTRACT Detecting semantic similarity between textual requirements is a crucial task for various natural language processing (NLP)-based requirements engineering (RE) applications. It is also challenging due to the nature of these requirements, which are written in natural language (NL), include domain knowledge, and often follow pre-defined templates that contain duplicated words. Recently, deep neural networks (DNNs) have shown promising results in measuring semantic similarity between texts. Siamese neural networks (SNNs), a class of DNNs, are widely used for measuring similarity between various data types, demonstrating their capability and independence of language and domain. Nevertheless, SNNs have a limited use in measuring semantic requirements similarity (SRS). In this paper, a novel metric-based learning method is proposed using SNNs that combines a sentence Transformer model (LLM) and long short-term memory (LSTM) networks with a backward network layer to measure semantic similarity between pairs of requirements. The proposed method is evaluated on an annotated SRS dataset that was built based on public datasets (i.e., PROMISE and PURE) and compared with other state-of-the-art methods (i.e., fine-tuning and zero-shot methods) using accuracy, precision, recall, and F1-score classification metrics. The results show that the proposed method achieved an accuracy of 95.42% and an F1-score of 95.71%, outperforming the state-of-the-art methods.

INDEX TERMS Artificial intelligence for requirements engineering, large language models, long short-term memory networks, requirements, requirements engineering, requirements similarity, semantic requirements similarity, Siamese neural networks, similarity, transformer models.

I. INTRODUCTION

Requirements engineering (RE) is the first and most important phase in the software development life cycle (SDLC), playing a significant role in software success. Errors discovered during the RE phase are less costly to fix than those discovered later, as all subsequent phases are based on the decisions made during this phase. Typically, requirements engineers need to elicit, understand, analyze, validate, and manage requirements that are ultimately documented in a software requirements specification [1], [2].

One of the key fundamental tasks applied for analyzing [3], documenting, and managing [4] requirements during the RE phase is detecting semantic requirements similarity (SRS). Such a task serves as the foundation for conducting further tasks, such as merging, grouping, or eliminating requirements, which then contribute to enhancing the quality of both the RE tasks (e.g., analysis and traceability [3], [4]) and final deliverables (e.g., requirements documents [2], [5], [6]).

Despite the importance of detecting SRS, manual identification of requirements similarity is tedious and challenging. This difficulty arises for several reasons. First, requirements are commonly written in natural language (NL) [2], [7], which is inherently ambiguous, complicated, and inconsistent [2], [8]. Second, requirements sentences are cross domain-specific, often written in a simple and concise form, with some common words (e.g., "system," "shall," "able,"

The associate editor coordinating the review of this manuscript and approving it for publication was Jolanta Mizera-Pietraszko.

© 2024 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

"software") that are frequently repeated across different systems' requirements [2], [6]. Third, the identification of semantic similar requirements requires domain expertise to detect semantic similarity between requirements [4] precisely. For example, consider the following requirements: 'The system shall be capable of printing user reports in various formats,' and 'The system shall execute all printing functions within 5 seconds.' While these requirements may appear similar to those of a novice, an expert would discern distinct categorizations, as they fall under different categories and require different implementation and testing techniques.

Several methods have been proposed to automatically measure the semantic similarity between pairs of text, such as knowledge-based (e.g., Wu & Palmer), corpus-based (e.g., Latent Semantic Analysis), and lexical string-based (e.g., Jaccard similarity) methods [9], [10]. In recent years, deep neural networks (DNNs) have shown promising results in measuring semantic similarity, outperforming other techniques such as TF-IDF and Word2Vec, and have resisted earlier attempts to use advanced natural language processing (NLP) techniques for automated processing [11], [12], [13].

Siamese neural networks (SNNs) are a class of DNNs architectures introduced in 1994 [14] data [15], [16]. SNNs consist of two identical networks, each of which takes a single pair of inputs. These networks work in parallel and share the same weights, parameters, and configurations that are updated during the model training [11], [17]. Compared to other DNNs architectures, SNNs require less training data and execution time, and they are less susceptible to overfitting, making them particularly suitable for semantic textual similarity (STS) applications, even with limited labeled datasets [15], [16].

SNNs have been used in applications such as face verification [18], image recognition [19], signature verification [14], and STS [11], [17]. For STS, SNNs have been widely used in applications involving pair-wise similarity detection, where the output of SNNs provides a measure of semantic similarity between pairs of inputs, demonstrating their language and domain independence [4], [10]. Researchers have turned their attention to exploring the capabilities of SNNs architectures in the context of STS applications [15], [16], offering the potential to enhance the accuracy, robustness, and scalability of these applications [11], [15], [16], [19]. Despite these advantages, the application of SNNs in textual RE tasks still needs to be explored [20], [21].

In the literature, studies on automating the detection and measurement of semantic textual similarities between requirements are few, and most were primarily designed to fulfill other RE tasks, such as requirements classification [3] and traceability [4]. Therefore, such studies need to provide more insight into the challenges and optimal solutions for accurately measuring requirements similarity. Additionally, these studies primarily relied on traditional corpus-based methods to calculate the similarity between words, such as word frequency-based techniques (e.g., part-of-speech word tagging) [22], term-based matching [23], and encoding requirements datasets using an extensible markup language (XML) rather than handling raw textual input directly [24]. These traditional methods and techniques are labor-intensive and time-consuming, lacking the efficiency and scalability offered by advanced deep learning (DL) methods.

Motivated by the promising performance of SNNs in measuring semantic textual similarity and the lack of advanced NLP and DL techniques in RE, we proposed a metric-based learning method for SRS detection. This method includes the architecture of the model by integrating SNNs with sentence Transformer large language models (LLMs) and bidirectional recurrent DNNs (Bi-LSTMs) components, as well as the use of SNNs in measuring requirements similarity.

Bidirectional long short-term memory (Bi-LSTM) networks [38] are integrated into our proposed method because they capture contextual information from both directions in a sequence. Unlike traditional LSTMs, which process information in a single direction, Bi-LSTMs analyze data from past and future contexts. This bidirectional processing enhances the capture of nuanced relationships and dependencies, leading to a more accurate semantic similarity measurement [25], [26]. Additionally, we incorporate a state-of-the-art sentence Transformer model to generate initial embeddings for the requirements sentences. Transformer models are highly effective at handling sequential data and capturing complex semantic relationships through attention mechanisms [27]. By combining SNNs with Bi-LSTMs and Transformer model, our method benefits from enhanced contextual analysis and sophisticated semantic understanding, significantly improving the accuracy and effectiveness of requirements similarity measurement.

The effectiveness of the proposed method was evaluated against various embedding LLMs and different state-of-the-art methods like fine-tuning and zero-shot learning (ZSL). The method was evaluated on requirements data collected from PROMISE dataset [28] and PURE dataset [29]. The results show that the proposed method outperforms other state-of-the-art methods, achieving an accuracy of 95.42%, precision of 96.30%, recall of 95.12%, and F1-score of 95.71%.

The key contributions of this study can be summarized as follows:

- Using SNNs in conjunction with LLMs and Bi-LSTM to measure semantic similarity between pairs of textual requirements, which have been trained on a small annotated dataset.
- Conducting empirical experiments to find the best embedding model for the proposed SNNs method and to empirically compare this method with state-of-the-art methods (fine-tune and ZSL).

The rest of this paper is structured as follows. Section II provides an overview of related works. Section III describes the proposed method. Section IV outlines our experimental methodology. Section V presents and analyzes the experimental results. Section VI discusses the research findings and the validity threats. Section VII concludes the paper.

II. RELATED WORK

Many methods have been proposed to detect similarities among software requirements artifacts. However, some of these methods are not designed primarily to measure requirement similarity, but rather to support other RE tasks, such as traceability [4], [7], [23], [24], [30], reusability [2], [31], [32], [33], [34], and classification [3], [5], [35]. Other methods are intended to measure relatedness, not similarity [2], [4], [24], [30], [31], [32]. Relatedness measures the correlation between pairs of requirements; related requirements are not substitutes, whereas similar requirements could be substitutable. For example, "receive" and "send" are related, but "receive" and "get" are similar. These works include studies conducted by Reddivari and J. Wolbert [23] and Alhoshan et al. [24], which measured relatedness using frequency-based methods.

The methods used to measure similarity in the related studies include lexical-based [33], [34], machine learning (ML)-based [5], DL-based [4], and frequency-based [2], [23], [24], [30], [31], [32] methods. These methods often require software requirements to be written in specific templates or heavily depend on external data. However, despite the variety of methods, there is a lack of application of recent DL techniques, such as Transformer-based models, in measuring requirements similarity.

Generally, few studies have used Transformer-based methods to automate RE tasks with less data in the context of RE. Examples of RE tasks automated with Transformer models include traceability [36], [37], requirements feature-based classification [38], [39], requirements ambiguity and conflicts classification [40], [41], and requirements quality identification and classification [8], [21], [42], [43], [44], [45] tasks. Although these studies show promising contributions, they have not explicitly proposed automating the SRS task. Additionally, there is a prevalence of using BERT-based models [46], with a lack of exploration of other recent advanced LLMs such as GPT-N [47], GPT-3 [48], and GPT-4 [49], XLNet [50], and USE [13] in this context. Models like MiniLM [51], T5 [52], SBERT [12], RoBERTa [53], GPT-1 [47], and GPT-2 [47] have been less frequently used in the RE domain [8], [36], [41], [42], [47].

Similarly, studies focusing on SNN and metric-based learning architectures are limited within the RE field. Although some studies reference using Siamese networks, they apply them differently and use traditional models such as GloVe for generating vector representations (embeddings) of words [39]. Other works include using Siamese networks (such as the Siamese MiniLM embedding model) in evaluation as part of a broader assessment without proposing them as primary models [42].

In summary, few related works primarily propose to measure semantic similarity

In summary, few related works primarily propose to measure semantic similarity between textual requirements, and there is a lack of using SNN methods or advanced pre-trained LLMs and Bi-LSTM networks for the same purpose. Motivated by these gaps, our work: 1) employs a Transformer-based method with advanced pre-trained LLMs for requirement embeddings, 2) applies a Siamese two-level DL method for detecting requirements similarity, and 3) provides more insight into developing and evaluating the semantic similarity between requirements.

III. PROPOSED METHOD

A. METHOD OVERVIEW

The proposed method utilizes deep SNN architecture for measuring pair-wise SRS. Fig. 1 shows an overview of the proposed method. The input is a pair of textual requirements, while the output is the label that indicates whether the two requirements are similar or dissimilar. The method consists of four main stages: pre-trained model embeddings, domain-specific Bi-LSTM embeddings, concatenation Bi-LSTM, and similarity prediction stages. The following paragraphs illustrate each stage separately.

1) PRE-TRAINED MODEL EMBEDDINGS STAGE

Each pair of the input requirements is tokenized to generate the requirement embeddings using a pre-trained Transformer model (LLM). These embeddings convert text input into numerical vectors, capturing the general semantics of the text. The pre-trained LLMs are often trained on a large corpus of general documents such as Wikipedia [44]. Thus, the generated embeddings at this stage are high-dimensional and include non-domain-specific information. Training embeddings on general data can reduce accuracy when dealing with domain-specific data [54], [55]. Therefore, the proposed method includes an additional stage (i.e., stage 2) to enhance the accuracy of requirement representations.

2) DOMAIN-SPECIFIC Bi-LSTM EMBEDDINGS STAGE

This stage represents the core architecture of SNNs: two identical DNNs, each of which learns a representation of an input vector, both sharing the same weights and working in parallel. In particular, we employed a recurrent DNN architecture (i.e. Bi-LSTM) to learn domain-specific semantic embeddings of the requirements. This results in two separate Bi-LSTM vectors representing the semantic information of the two requirements.

3) CONCATENATION Bi-LSTM STAGE

The two separate Bi-LSTM vectors generated from the previous stage are concatenated to combine both forward and backward contextual information for each requirement, resulting in a more robust and informative representation [25]. The concatenated vector is fed into a backward Bi-LSTM layer to enhance the concatenated feature representation further, enabling the method to extract and compare the semantics of requirement pairs, thereby enhancing accurate similarity detection.

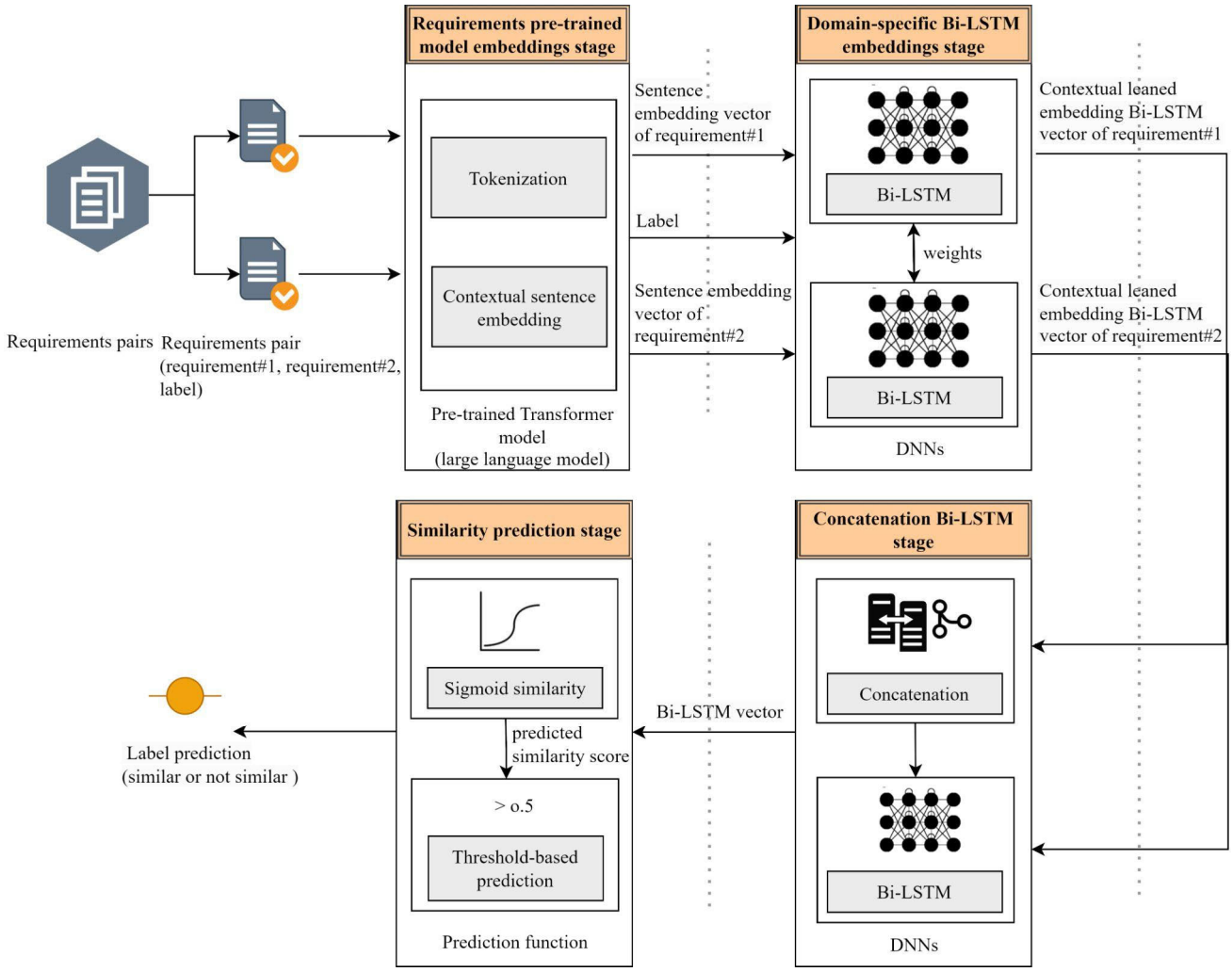

FIGURE 1. The proposed method for SRS detection in requirements textual data.

4) SIMILARITY PREDICTION STAGE

In this stage, the semantic similarity is calculated using the sigmoid similarity network method [56], which has been widely utilized in various semantic similarity domains to assess the similarity between different pairs of data points. The output of the sigmoid network is the probability (ranging from 0 to 1) that represents the similarity between pairs of the input requirements, with 0 indicating dissimilarity and 1 indicating similarity. To convert the probability into binary labels (similar and dissimilar), the probability score is fed into a threshold-based prediction function, with a decision threshold (δ) of 0.5 (i.e., $\hat{y} = 1$, if $p(y') > 0.5$). The output of this stage, and the whole method, is the label that indicates whether the two requirements are similar or dissimilar.

Algorithm 1 outlines the process used by our proposed method. The input to the algorithm is pairs of requirements (r_i, r_j), which go through the following steps.

1) *Pre-processing (Tokenization):* Each pair of requirements (r_i, r_j) is first tokenized by breaking down each requirement into smaller units (tokens).
2) *Sentence Embedding Generation:* Tokens from each requirement r_i and r_j are used to generate sentences embeddings using a pre-trained LLM model.
3) *Domain-Specific Vector Creation:* The sentence embeddings are then passed through DNN layers. These embeddings are refined into domain-specific vectors v_i and v_j using Bi-LSTM networks that learn and incorporate specialized knowledge relevant to the RE field.
4) *Similarity Score Computation:* The domain-specific vectors v_i and v_j are compared to compute and normalize a similarity score ss using an activation function.
5) *Label Assignment:* Based on the similarity score ss and a pre-defined threshold δ, a similarity label sl is assigned to each pair of requirements.

B. DL MODELS USED BY THE PROPOSED METHOD

The proposed method employs pre-trained LLMs and Bi-LSTMs as the main DL models. Below, we introduce these models.

Algorithm 1 Semantic Requirements Similarity Detection

Input:
Requirement Pairs: $R = \{(r_{i1}, r_{j1}), (r_{i2}, r_{j2}), (r_{i3}, r_{j3}), \ldots, (r_{in}, r_{jn})\}$
Threshold: δ
Output:
Similarity Label: $L = \{sl_1, sl_2, sl_3, \ldots, sl_n\}$, where $sl \in \{0$ (dissimilar), 1(similar) $\}$
Similarly Score: $S = \{ss_1, ss_2, ss_3, \ldots, ss_n\}$, where $ss \in [0, 1]$
Steps:
// Predict and assign similarity scores:
def *TrainedSiameseModel* (r_i, r_j):{
// 1. Pre-process and Tokenize:
for each pair (r_i, r_j) in R:
t_i = *LLM.tokenizer*(r_i) // Extract tokens of r_i
t_j = *LLM.tokenizer*(r_j) // Extract tokens of r_j
// 2. Generate Sentence Embedding:
e_i = *LLM.model*(t_i) // Generate sentence embedding of t_i
e_j = *LLM.model*(t_j) // Generate sentence embedding of t_j
// 3. Create Domain-Specific Vectors:
v_i = *Bi-LSTM*(e_i) // Generate domain-specific vector of e_i
$v_j = Bi - LSTM(e_j)$ // Generate domain-specific vector of e_j
// 4. Compute Similarity Score:
$ss = sigmoid(v_i, v_j)$ // Generate similarity score of (v_i, v_j)
S.add(ss) //append ss to S
return*S* //similarity scores}
// 5. Assign Labels:
// Label the requirement pairs:
def *AssignLabel (S)*:{
for each similarity score ss in S:
// Assign Label to $(\acute{r}_i, \acute{r}_j)$ based on ss and threshold (δ)
if $(ss > \delta)$, *then* $(sl = 1)$ //similar
else, $(sl = 0)$ //dissimilar
L.add(sl) //append sl to L
return*L* //similarity labels}

1) LLMs

LLMs are advanced DL models designed to understand human-like text through extensive training datasets. They are capable of performing a wide range of NLP tasks, including the generation of sentence embedding [47], [48], [57]. LLMs are built on Transformer architecture, revolutionizing NLP research with the seminal paper "Attention Is All You Need" [27] in 2017. Initial LLMs were developed based on Transformer decoder and encoder layers. Subsequent models such as XLNet, RoBERTa, and T5 have further advanced the field. XLNet [50] and RoBERTa [53] improve upon BERT's architecture, while T5 [52] is an encoder-decoder-based model, demonstrating that the performance of LLMs can be enhanced simply by increasing the model size [58]. This insight has led to the development of large-scale LLMs like GPT-3 (175B) [48], which encompass a large number of parameters. LLMs provide substantial background knowledge to NLP-based tasks, thus eliminating the need to train models from scratch [59], [60].

2) LLMs FOR GENERATING REQUIREMENT EMBEDDINGS

LLMs can be used to generate word-based embeddings and sentence-based embeddings to represent a single requirement sentence. Word-based embeddings capture the meaning of each word in context by representing individual words in a continuous vector space. These are commonly used in existing studies, with BERT-based models [46] being the most utilized in this group. Sentence-based embedding models represent the whole sentence in vector space. SBERT [12] and MiniLM [51] models, both based on the BERT architecture, are predominant in current research within this group.

3) Bi-LSTM

Bi-LSTM is an advanced extension of the unidirectional LSTM architecture, initially introduced by Schuster and Paliwal in 1997 [61]. Bi-LSTM [61] is a type of recurrent neural network (RNN) DL architecture that processes sequential data input in both forward and backward directions [62]. This bidirectional processing makes it particularly suitable for handling sequential textual data, allowing it to effectively capture the semantic properties of short texts [26], [62]. By considering information from past and future contexts, Bi-LSTM enhances its understanding of text input sequences, enabling it to capture intricate patterns and dependencies more comprehensively [4].

IV. EXPERIMENTAL DESIGN

A. RESEARCH QUESTIONS (RQs)

We aim to evaluate the effectiveness of the proposed method in measuring the similarity between requirements using different pre-trained embedding models (LLMs) to identify the best one for use and to compare the method with other state-of-the-art methods (i.e., fine-tuning and zero-shot) to validate its efficacy. In other words, the evaluation of the method seeks to answer the following research questions:

- **RQ1:** Which pre-trained sentence embedding model (LLM) provides the best performance for embedding the requirements input in the proposed method?
- **RQ2:** How effective is the proposed method in comparison with the fine-tuning LLM method?
- **RQ3:** How effective is the proposed method in comparison with the embedding-based ZSL method?

B. RESEARCH PROCESS

To answer the RQs, we conducted three experimental studies, each aiming to answer a single research question.

1) STUDY 1: EMPIRICAL COMPARISON OF REQUIREMENT EMBEDDINGS MODELS USED IN THE PROPOSED METHOD (RQ1)

This study empirically compares several sentence embedding models to determine which demonstrates the best performance. Specifically, we empirically compared the performance of thirteen pre-trained Transformer models. Six of these models are word-based sentence embedding LLMs that capture the meaning of words based on their context: BERT [46], RoBERTa [53], GPT-2 [47], GPT-1 [47], T5 [52], and XLNet [50]. The other seven are sentence-based sentence embedding LLMs that consider the context of all words within the sentence: SBERT [12], RoBERTa [53], MiniLM [51], LangChain [63], GPT-3 [48], GPT-4 [49], and USE [13]. Among the

sentence-based models, two are not free to use (GPT-3 and LangChain).

We have selected BERT-based models because of their popularity in related studies and suitability for pair-wise classification research problems [43]. Additionally, we included other LLMs such as GPT-2 and GPT-3 by OpenAI, USE, T5, and XLNet in our study because their encoders are suitable for encoding requirements sentences and their less frequent application in the related pair-wise classification research problems [8], [41], [42], [47].

2) STUDY 2: COMPARISON OF THE PERFORMANCE OF FINE-TUNED LLMs AGAINST THE PROPOSED METHOD IN MEASURING SEMANTIC REQUIREMENTS SIMILARITY (RQ2)

This study aims to empirically compare the performance of fine-tuned LLM models against the proposed method in measuring requirements similarity. Using fine-tuned models has shown promising results in recent related studies on requirements semantic similarity measures [45]. In particular, we compared the performance of six LLMs (BERT [46], RoBERTa [53], GPT-2 [47], GPT-1 [47], T5 [52], and XLNet [50]) that are fine-tuned using a requirements dataset against the proposed method.

3) STUDY 3: COMPARISON OF ZSL AGAINST THE PROPOSED METHOD IN MEASURING SEMANTIC REQUIREMENTS SIMILARITY (RQ3)

This study aims to empirically compare the performance of the proposed method with zero-shot models, a recent advance method in NLP that requires no training (i.e., labeled dataset) to perform specific tasks such as classification. The ZSL method uses only pre-trained models to generate requirement embeddings and cosine similarity to calculate their similarity. In particular, this study compares the performance of the thirteen pre-trained Transformer models: six word-based models (BERT [46], RoBERTa [53], GPT-2 [47], GPT-1 [47], T5 [52], and XLNet [50]) and seven sentence-based models (SBERT [12], RoBERTa [53], MiniLM [51], LangChain [63], GPT-3 [48], GPT-4 [49], and USE [13]).

C. DATASET PREPARATION

Due to the lack of available requirements datasets consisting of pairs of requirements annotated as similar or dissimilar [36], [54], we built one called RP4RSDataset. This dataset is available for others to use in related contexts. The requirements statements of the dataset were collected from two widely used datasets in RE research: the PROMISE dataset [28] and the PURE dataset [29]. The dataset consists of 1530 pairs, with 50% of these pairs annotated as similar (765 pairs) and the other 50% as dissimilar (765 pairs).

Two external annotators were asked to label pairs of the requirements as 'similar' or 'dissimilar' to annotate the requirements. These annotators are employed as software engineers (or those with a software engineering background) to ensure that the context of software development is considered when measuring requirements similarity. A third annotator (the first author) was included in the annotation process to make the final decision using majority voting to resolve discrepancies among the annotators.

TABLE 1. Data splitting.

Subset		No of pairs	Percentage (%)		
Development set	Train set	979	80%	80%	64%
	Valid set	245		20%	16%
Testing set	Test set	306	20%		20%

D. EXPERIMENTAL IMPLEMENTATION SETUP

All experiments were implemented using Python in Google Colab with GPU acceleration. The key libraries to develop the proposed method include TensorFlow, Keras, and PyTorch. Scikit-learn was used for performance measurement and cosine similarity computation. The pre-trained models generated requirement embeddings were imported from the HuggingFace and OpenAI libraries. HuggingFace is a well-known NLP community repository offering open-source AI-powered pre-trained language models and other language resources. At the same time, OpenAI is renowned for its advanced AI research, which offers paid users access to open pre-trained GPT LLMs.

The settings for these models, such as number of epochs and batch size, were chosen based on several preliminary experiments. The thresholds for making classification decisions were determined based on initial preliminary experiments on a random on a random requirements sample.

E. PERFORMANCE MEASURES

To evaluate the performance of our method, we used standard evaluation metrics: Accuracy (A), Precision (P), Recall (R), and F1-score (F1). Additionally, we have computed experimental execution time in terms of training and prediction time. F1-score (F1) and accuracy (A) are metrics used in this study for comparing results. F1-score is the harmonic mean of precision and recall, considering both false positives (FP) and false negatives (FN). Accuracy is the percentage of correct predictions relative to the total number of predictions. Since the data is balanced, accuracy is a valuable metric to use for binary classification tasks because it straightforwardly measures the overall correctness of predictions [64], [65].

F1-score and accuracy measurement metrics are used because they provide a comprehensive evaluation of the model's performance and are widely adopted in the field [1], [54]. Improvements in accuracy and F1-score are crucial as they directly impact the effectiveness and reliability of automated SRS classification. A high F1-score indicates a better balance between precision and recall, ensuring that both FP and FN are minimized in measuring the similarity

TABLE 2. Results of applying different sentence embedding models in the proposed Siamese-based method.

		Model	Training and prediction time	Accuracy (A)	Precision (P)	Recall (R)	F1-score (F1)
1	Word-based LLM	**RoBERTa**	26min 27.19s	**95.42%**	**96.30%**	**95.12%**	**95.71%**
2		XLNet	31min 41.3s	93.79%	93.94%	94.51%	94.22%
3		BERT	26min 49.3s	94.12%	95.06%	93.90%	94.48%
4		GPT2	29min 27s	91.18%	92.02%	91.46%	91.74%
5		GPT1	27min 38.5s	93.46%	93.90%	93.90%	93.90%
6		T5	15min 28s	81.37%	86.39%	77.44%	81.67%
7	Sentence-based LLM	GPT3	27min 16.8s	**94.12%**	**92.07%**	**96.79%**	**94.38%**
8		RoBERTas	29min 7.9s	73.53%	73.97%	71.52%	72.73%
9		USE	3min 51.75s	82.43%	84.72%	84.72%	83.56%
10		SBERT	3min 6.24s	92.81%	92.05%	93.29%	92.67%
11		LangChain	10min 21s	87.25%	85.81%	88.67%	87.21%
12		GPT4	4min 0.66s	83.66%	82.61%	85.81%	84.18%
13		MiniLM	3min 9.25s	84.97%	84.91%	85.99%	85.44%

between requirements. High accuracy reflects fewer errors in detecting similarity between pairs of requirements, showcasing the model's consistent ability to make correct predictions.

We considered the train-validation-test split method in all experiments to evaluate performance. The dataset was split into three subsets: the training set (64%), the validation set (16%), and the testing set (20%), following the 80/20 rule. Table 1 gives a detailed breakdown of the count of each type of requirement in the training, valid, and test sets.

We also visualized the data distribution of correct and incorrect similarity predictions on the test data. These visualizations were helpful summarizing error rates between the two labels. They were plotted using matplotlib and seaborn libraries.

V. EVALUATION RESULTS

This section presents the results of a series of experiments that address the research questions (RQs).

A. SENTENCE EMBEDDING MODELS FOR THE PROPOSED METHOD (RQ1)

Table 2 shows the overall results of applying several pre-trained models to generate requirement embeddings for the proposed method. The models are grouped into word-based level and sentence-based level categories. As the table indicates, among the word-based models, RoBERTa achieved the highest accuracy (95.42%) and F1-score (95.71%), followed by BERT, XLNet, and GPT-1 with accuracies of 94.12%, 93.79%, and 93.46%, and F1-scores of 94.48%, 94.22%, and 93.90%, respectively. For the sentence-based models, GPT-3 demonstrated the highest performance with an accuracy of 94.12% and an F1-score of 94.38%, followed by SBERT with an accuracy of 92.81% and an F1-score of 92.67%.

When comparing word-based and sentence-based models, word-based models generally showed higher performance, with RoBERTa being the top-performing model. Consequently, RoBERTa will be used in the proposed method.

In terms of the time taken for training and predicting labels, sentence-based models generally require less time to train and predict labels than word-based models. SBERT and MiniLM were the fastest sentence-based models, taking less than 4 minutes, while the GPT-2 word-based model required about 30 minutes to train and predict the requirement pairs. RoBERTa, the model used in the proposed method, took 26 minutes and 27.19 seconds, making it the second fastest word-based model after T5.

Fig. 2 shows the visualization of the predictions for each class, particularly highlighting the error rates associated with each label. The figure illustrates that RoBERTa achieved the lowest error rate with the fewest mistakes across both labels compared to other models, emphasizing the choice of RoBERTa as the embedding model for the proposed method.

B. COMPARING THE PROPOSED SIAMESE-BASED METHOD WITH FINE-TUNING LLM METHOD (RQ2)

Table 3 shows the results of applying the fine-tuning method to classify pairs of requirements as similar or dissimilar. As the table shows, the highest performance is achieved by applying RoBERTa, with an accuracy of 88.89% and an F1-score of 89.57%. This is followed by GPT-1 with an accuracy of 88.56% and an F1-score of 89.23%. Regarding execution time, RoBERTa took the shortest time for training and prediction at 19 minutes and 24.24 seconds.

In comparison with the proposed method, as the table shows, the proposed method outperforms the fine-tuned models, including RoBERTa. However, it took longer time to execute compared to the RoBERTa, BERT, and T5 fine-tuned models.

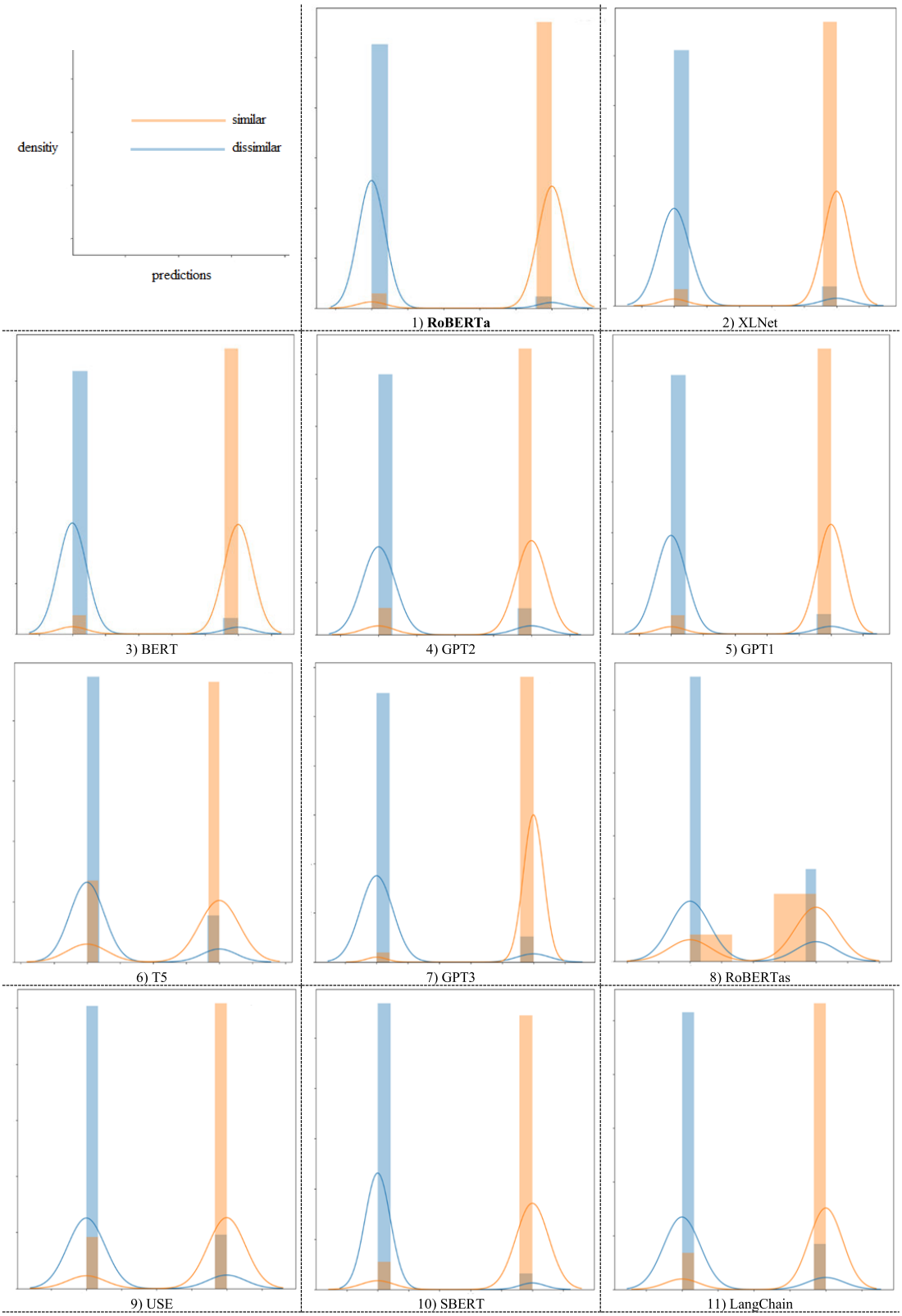

FIGURE 2. Visualizing distributions of Siamese-based models' predictions. Bars show the distributions of predicted values for each label (similar and dissimilar), displayed side by side. Curves represent density of predictions. This visualization provides insight into the error rates of the predicted outputs generated by these models.

Fig. 3 visualizes the error rates associated with each label. It shows that RoBERTa achieved the lowest error rate, making the fewest mistakes across both labels compared to other models. In contrast, T5 showed the

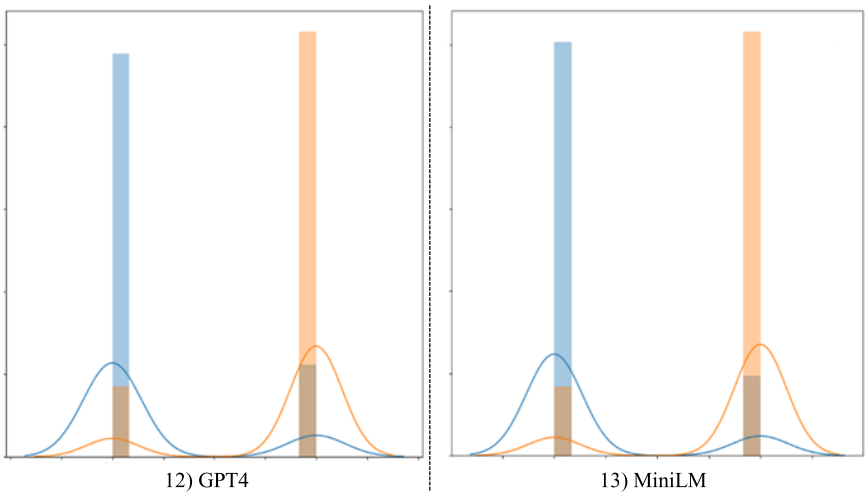

FIGURE 2. ***(Continued.)*** **Visualizing distributions of Siamese-based models' predictions. Bars show the distributions of predicted values for each label (similar and dissimilar), displayed side by side. Curves represent density of predictions. This visualization provides insight into the error rates of the predicted outputs generated by these models.**

TABLE 3. **Results of fine-tuning different LLMs.**

		Model	Training and prediction time	Accuracy (A)	Precision (P)	Recall (R)	F1-score (F1)
		The proposed method	26min 27.19s	**95.42%**	**96.30%**	**95.12%**	**95.71%**
1	LLM	RoBERTa	19min 24.24s	**88.89%**	**90.12%**	**89.02%**	**89.57%**
2		XLNet	30min 31.6s	85.95%	83.06%	82.68%	87.61%
3		BERT	20min 35.66s	88.24%	89.45%	87.12%	88.75%
4		GPT2	30min 27s	76.80%	74.35%	86.59%	80.00%
5		GPT1	31min 10s	88.56%	89.51%	88.96%	89.23%
6		T5	20min 41.41s	61.11%	59.18%	88.41%	70.90%

highest error rate, indicating the most mistakes across both labels.

C. COMPARING THE PROPOSED SIAMESE-BASED METHOD WITH EMBEDDING-BASED ZSL METHOD (RQ3)

Table 4 illustrates the results of applying the embedding-based ZSL method with various LLMs. As shown in the table, among sentence-based models, MiniLM achieved the highest accuracy of 63.01% and an F1-score of 68.20%, while SBERT achieved the highest F1-score of 69.41% and an accuracy of 59.74%. USE and GPT-4 closely followed with accuracies of 62.16% and 61.31%, and F1-scores of 63.93% and 65.22%, respectively. Sentence-based models generally outperformed word-based models, with MiniLM, USE, GPT-4, and SBERT demonstrating higher performance compared to word-based models such as BERT and XLNet achieving an accuracy of 50.00% and an F1-score of 66.67%. As shown from the table, the proposed method outperforms the ZSL models, including MiniLM and SBERT.

Regarding training and prediction times, sentence-based models generally require less time than word-based models. RoBERTa and GPT-3 showed the longest times among the sentence-based models, with RoBERTa requiring over 2 hours and GPT-3 needing more than 30 minutes. Among the word-based models, GPT-1 and RoBERTa took over 17 minutes to train and predict the requirement pairs. XLNet, a word-based model, completed the task in 5.42 seconds, making it the fastest among the ZSL models.

Fig. 4 illustrates that most of the requirement pairs are classified under the 'similar' label, as observed in BERT and GPT-2. Half of the dataset pairs are dissimilar pairs but were often detected as similar. One primary reason for this is common words and sentence structure within requirements sentences. Since this method required no training dataset, the results show that measuring requirements similarity using this method is challenging. From our observations, the ZSL models primarily measure relatedness rather than similarity, resulting in higher cosine similarity scores for requirements that share common words despite potential dissimilarities in other aspects.

VI. DISCUSSION

This section presents the research's key findings and the factors affecting their credibility and applicability, known as validity threats.

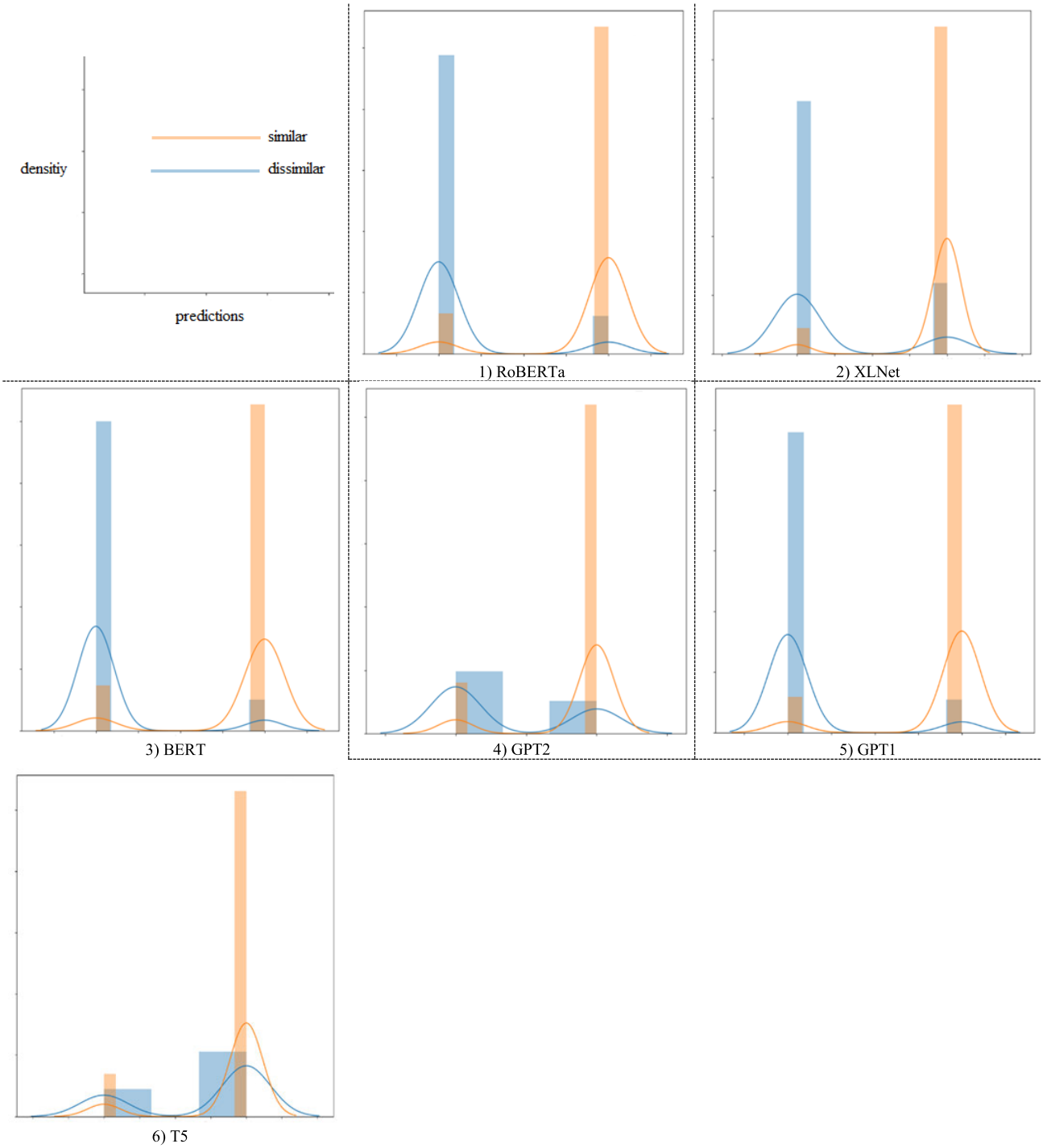

FIGURE 3. Visualizing distributions of fine-tuned models' predictions. This visualization illustrates the distributions of predicted values for two labels (similar and dissimilar), displayed side by side. Each curve represents the density of predictions. This visualization provides insight into the error rates of the predicted outputs generated by these models.

A. RESEARCH MAIN FINDINGS

Based on the above experimental results, this section summarizes the main findings and general observations from our experiments.

- SNNs show their effectiveness in measuring semantic similarity between requirements with accuracy and F1-score exceeding 95%, outperforming state-of-the-art methods (i.e., fine-tuning and zero-shot learning methods).
- In the proposed method, integrating Bi-LSTM DNN layers with SNNs yields effective results in measuring SRS. This approach eliminates the need for additional DNN layers, such as convolutional and pooling, while maintaining better performance. The combination of

TABLE 4. Results of applying embedding-based ZSL method using different sentence embedding models.

		Model	Training and prediction time	Accuracy (A)	Precision (P)	Recall (R)	F1-score (F1)
		The proposed method	26min 27.19s	**95.42%**	**96.30%**	**95.12%**	**95.71%**
1	Word-based LLM	RoBERTa	17min 46.13s	50.00%	50.00%	100.0%	66.67%
2		XLNet	5.42 s	50.00%	50.00%	100.0%	66.67%
3		BERT	8min 39.28s	50.00%	50.00%	100.0%	66.67%
4		GPT2	4min 46.38s	50.00%	50.00%	100.0%	66.67%
5		GPT1	18min 27.36 s	52.55%	51.48%	88.76%	65.16%
6		T5	1min 50.34s	53.20%	51.92%	86.41%	64.87%
7	Sentence-based LLM	GPT3	37min 55.93s	50.00%	50.00%	100.0%	66.67%
8		RoBERTas	2h 23min 32.28s	50.00%	50.00%	100%	66.67%
9		USE	3min 49.64s	62.16%	**61.07%**	67.06%	63.93%
10		SBERT	7min 8.96s	59.74%	55.96%	91.37%	**69.41%**
11		LangChain	6min 49.4s	50.00%	50.00%	100.0%	66.67%
12		GPT4	2min 8.28s	61.31%	59.23%	72.55%	65.22%
13		MiniLM	2min 39.05s	**63.01%**	59.80%	79.35%	**68.20%**

SNNs with Bi-LSTM layers, alongside the advanced pre-trained Transformer model (RoBERTa), highlights our method's novel aspect and underscores this integration's significant contribution for enhanced SRS detection.

- The word-based RoBERTa model is the most effective pre-trained LLM (sentence Transformer model) with the proposed architecture for representing all requirement pairs in the dataset. RoBERTa also yielded stronger results when using the fine-tuning method.
- For the SNNs method, RoBERTa and XLNet word-based LLMs and GPT-3 and SBERT sentence-based LLMs have emerged as four advanced models for the SRS detection task. These advanced models demonstrate their effectiveness in handling complex detection problems.
- Among sentence-based models, GPT-3 shows the best sentence-based LLM for the SNNs method. Despite being provided by OpenAI and not freely available for reuse, GPT-3 shows promising results. SBERT, among free sentence-based models, shows a good free alternative sentence-based LLM for representing requirement pairs.
- T5 is the worst LLM for the SRS detection task. In contrast, the study [42] found that the proposed fine-tuned T5-based models best suit its specific classification task.
- MiniLM and SBERT are sentence-based embedding models that show high performance in SRS: MiniLM with the ZSL method, and SBERT with the SNN-based method. MiniLM also demonstrated high results in a recent study that used the ZSL method for requirements classification [21].
- Sentence-based LLMs (such as MiniLM and SBERT) with a ZSL method yielded better results than word-based LLMs, indicating a preference for sentence-based models over word-based models in generating requirement embeddings. However, with SNNs method, word-based models perform better than sentence-based models.
- Among the three methods used (SNNs, fine-tuning, and ZSL), the ZSL method generally requires less execution time. However, it also shows the lowest performance in terms of accuracy and F1-score.
- Since the SNNs and fine-tuning methods are more effective than ZSL method in measuring requirements similarity, this highlights the importance of domain-specific training data in enhancing requirements similarity detectors and classifiers. Exploring the addition of various numbers of examples (i.e., few-shot) could influence the performance of ZSL, providing a trade-off between performance and execution time. This could be a focus for future research.
- SNNs require few data during training for effective SRS detection. The size of the annotated dataset is crucial for the SSN's performance. For example, the Siamese MiniLM model implemented in [51] showed considerable improvement when more data training data was available, and the study [42] showed that the proposed T5-based models demonstrated significant improvement with additional training data for domain-specific tasks for their task. Conducting more future experiments further to analyze the impact of data size on SNNs performance will be a focus for future research.
- We noticed that most pairs were detected as similar by applying ZSL. By analyzing the models' outcomes, we found that this method detects relatedness more than similarity. For example, since the structure of requirements sentences is similar and the requirements include common general words, the detected scores are nearly 1. As a result, almost half of the data detected as similar, approximately 765 pairs (50%), were incorrectly detected. This is why the recall measure reached 100% in many cases, indicating that all pairs were predicted as similar.

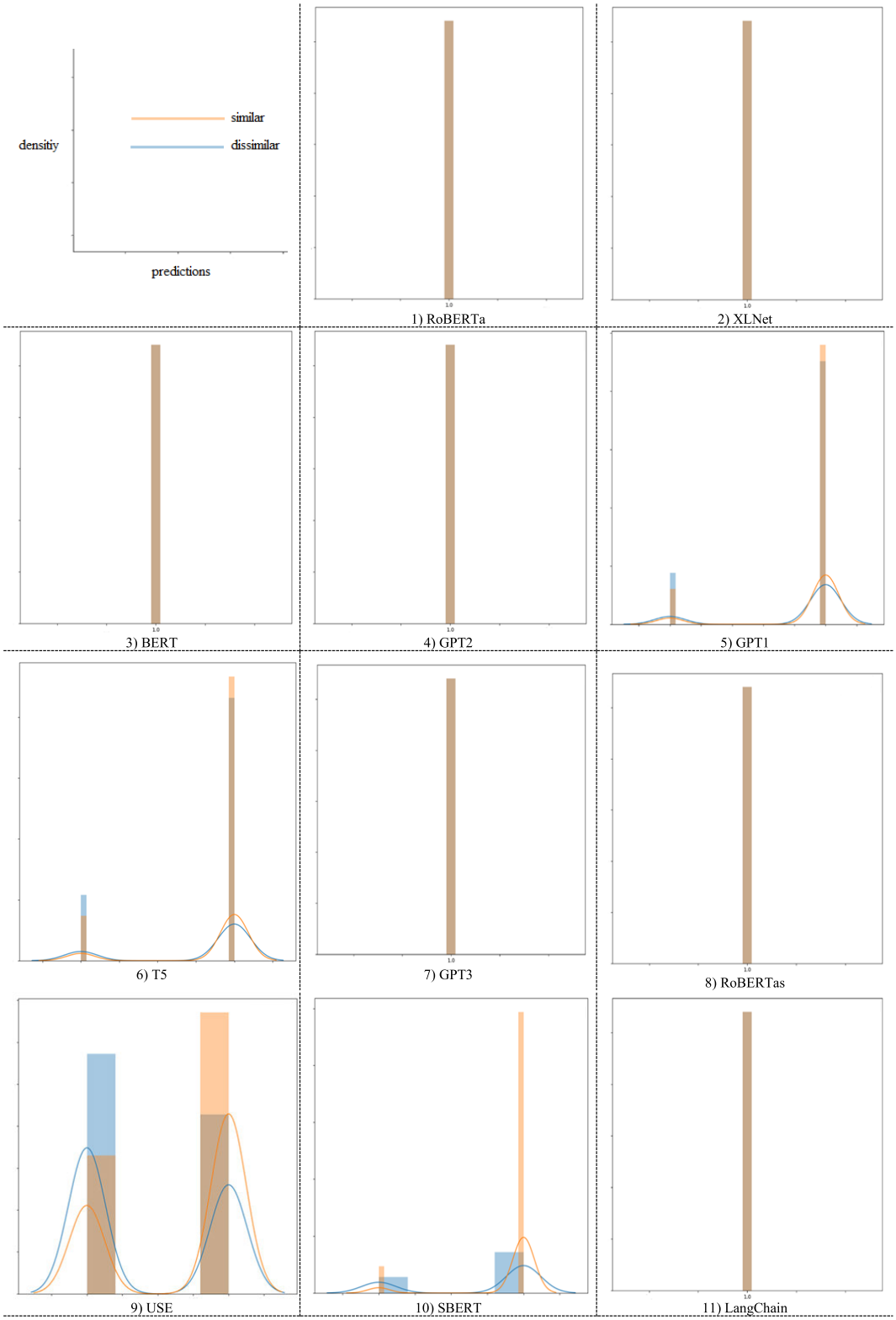

FIGURE 4. Visualizing distributions of ZSL models' predictions. This visualization illustrates the distributions of predicted values for two labels (similar and dissimilar), displayed side by side. Each curve represents the density of predictions. This visualization provides insight into the error rates of the predicted outputs generated by these models. The single bar indicates that all dissimilar data (represented by the blue bar) is predicted as similar (shown upon the orange bar), as shown by the BERT and GPT-2 models.

- One of the key potential reasons for predicting most of the pairs similar with ZSL is that the LLMs are trained on general-domain datasets, which reduces their accuracy when dealing with domain-specific data [54], [55].

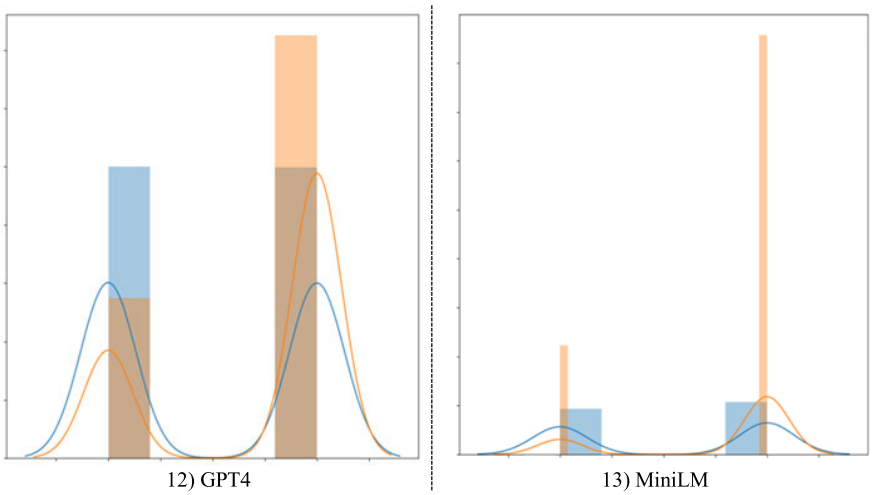

FIGURE 4. ***(Continued.)*** **Visualizing distributions of ZSL models' predictions. This visualization illustrates the distributions of predicted values for two labels (similar and dissimilar), displayed side by side. Each curve represents the density of predictions. This visualization provides insight into the error rates of the predicted outputs generated by these models. The single bar indicates that all dissimilar data (represented by the blue bar) is predicted as similar (shown upon the orange bar), as shown by the BERT and GPT-2 models.**

Applying ZSL with LLMs trained on domain-specific training datasets could improve the results of this method in detecting requirements similarity. This is another direction for future research.

- Most models built on the base of sentence-based LLMs for ZSL are faster than those built on the base of word-based LLMs for ZSL. XLNet took the shortest time (around 6 seconds).

B. THREATS TO VALIDITY

Validity threats in empirical software engineering research include external, internal, and construct validity [66]. Addressing these threats is crucial for analyzing validity, minimizing their impact, and ensuring that research findings are robust and applicable. Below, we discuss the threats in our research and how we have mitigated them.

1) EXTERNAL VALIDITY

One of the main threats to this study's external validity is due to the use of the PROMISE [28] and PURE [29] datasets. The requirements in the PROMISE dataset were provided by students, and the PURE dataset was created from 79 PDF files found on the Web, which might not be representative of real-world industrial requirements. The PROMISE and PURE datasets have been widely adopted as benchmarks for research in the RE community. To mitigate this threat, we used part of these datasets that were selected carefully to address the research questions with the help of domain experts.

2) INTERNAL VALIDITY

Another potential threat concerns the labeled dataset used to train and test the models. The researchers followed a specific procedure to mitigate the risk of bias in the label selection process. This involved independent labeling by domain experts, followed by majority voting to determine the final labels. Additionally, the researchers reported the obtained agreements between the experts, which were moderated in all the cases. This method was intended to ensure the reliability and objectivity of the expert-provided labels used in the study.

3) CONSTRUCT VALIDITY

Another threat to the study is related to construct validity, which relates to how well the measurements used in the study capture the intended constructs. To mitigate this potential threat, the measurements used to evaluate the methods are widely used measures for the classification task in the RE field [1], [21] and other domains [64], [65] (i.e., precision, recall, and F1-score). Additionally, the dataset was balanced; thus, we used accuracy to evaluate the methods [64], [65]. Besides the evaluation metrics, we visualized the predictions to provide a more comprehensive evaluation and analysis.

VII. CONCLUSION

This study proposes a method to automate semantic similarity measurement between pairs of requirements. The proposed method is based on SNN architectures, which have been less frequently used in the RE domain, as well as advanced LLM models. Specifically, the proposed method uses the advanced pre-trained Transformer model (i.e., RoBERTa) in conjunction with a Bi-LSTM deep neural network architecture, a novel combination within the RE domain, to the best of our knowledge.

More than 250 experiments were conducted, each built upon a combination of different settings such as method type, hyper-parameters, a specific LLM, a specific detection

task, and a specific similarity measurement calculation. Our findings showed that the proposed automated method outperformed other methods (fine-tuning and zero-shot learning) in terms of accuracy and F1-score, with an accuracy of 95.42% and an F1-score of 95.71%. The method has shown promising results and has the potential to contribute to the advancement of the SRS detection task significantly.

The findings of this study highlight new directions in the field of RE and similarity-based methods. For example, the effectiveness of SNNs combined with Bi-LSTM networks suggests that similar methods could be effective within or without the software engineering domain. Additionally, the proposed method, which demonstrated high accuracy, effectively measures requirements similarity. This supports its application to other RE tasks, such as requirements classification and identifying requirements duplication and incompleteness. It also reduces the need for manual-intensive work in performing these tasks or collecting data.

Future work will focus on four main directions:

- **Improving Performance:** Enhancing the proposed method by exploring more recent LLMs, such as GPT-5 and SemEval, and experimenting with additional combinations of DNN layers (e.g., convolutional neural networks (CNNs)).
- **Few-Shot Learning:** Investigating the impact of increasing the number of samples (shots) in measuring requirements similarity, leveraging few-shot learning (FSL) techniques. This method has shown promising results in measuring similarity for tasks such as image and short text classification.
- **Application to Other Software Engineering Tasks:** Exploring the proposed method's applicability to other software engineering tasks, such as requirements classification, including non-functional requirements (NFRs).
- **Domain-Specific Data:** Examining the effects of using domain-specific data to train LLMs with the proposed method, as well as other methods applied in this study (e.g., zero-shot method), to assess requirements similarity.

REFERENCES

[1] Q. A. Shreda and A. A. Hanani, "Identifying non-functional requirements from unconstrained documents using natural language processing and machine learning approaches," *IEEE Access*, early access, Jan. 19, 2021, doi: 10.1109/ACCESS.2021.3052921.

[2] M. Abbas, A. Ferrari, A. Shatnawi, E. P. Enoiu, and M. Saadatmand, "Is requirements similarity a good proxy for software similarity? An empirical investigation in industry," in *Requirements Engineering: Foundation for Software Quality*, F. Dalpiaz and P. Spoletini, Eds., Cham, Switzerland: Springer, 2021, pp. 3–18, doi: 10.1007/978-3-030-73128-1_1.

[3] M. Younas, D. N. A. Jawawi, I. Ghani, and M. A. Shah, "Extraction of non-functional requirement using semantic similarity distance," *Neural Comput. Appl.*, vol. 32, no. 11, pp. 7383–7397, Jun. 2020, doi: 10.1007/s00521-019-04226-5.

[4] J. Guo, J. Cheng, and J. Cleland-Huang, "Semantically enhanced software traceability using deep learning techniques," in *Proc. IEEE/ACM 39th Int. Conf. Softw. Eng. (ICSE)*, May 2017, pp. 3–14, doi: 10.1109/ICSE.2017.9.

[5] W. Guo, L. Zhang, and X. Lian, "Automatically detecting the conflicts between software requirements based on finer semantic analysis," 2021, *arXiv:2103.02255*.

[6] C. E. N. Gal-Chiş and B. Pârv, "MultiCoS—A requirements engineering tool," *Int. J. Softw. Eng. Knowl. Eng.*, vol. 28, no. 1, pp. 37–55, Jan. 2018, doi: 10.1142/s021819401850002x.

[7] M. A. Alfianto, Y. Priyadi, and K. A. Laksitowening, "Semantic textual similarity in requirement specification and use case description based on sentence transformer model," in *Proc. IEEE Int. Conf. Ind. 4.0, Artif. Intell., Commun. Technol. (IAICT)*, Jul. 2023, pp. 220–226, doi: 10.1109/IAICT59002.2023.10205769.

[8] A. T. Ray, B. F. Cole, O. J. P. Fischer, R. T. White, and D. N. Mavris, "AeroBERT-classifier: Classification of aerospace requirements using BERT," *Aerospace*, vol. 10, no. 3, p. 279, Mar. 2023, doi: 10.3390/aerospace10030279.

[9] A. Abo-Elghit, A. Al-Zoghby, and T. Hamza, "Textual similarity measurement approaches: A survey (1)," *Egyptian J. Lang. Eng.*, vol. 7, no. 2, pp. 41–62, Sep. 2020, doi: 10.21608/ejle.2020.42018.1012.

[10] J. V. A. de Souza, L. E. S. E. Oliveira, Y. B. Gumiel, D. R. Carvalho, and C. M. C. Moro, "Exploiting Siamese neural networks on short text similarity tasks for multiple domains and languages," in *Computational Processing of the Portuguese Language*, P. Quaresma, R. Vieira, S. Aluísio, H. Moniz, F. Batista, and T. Gonçalves, Eds., Cham, Switzerland: Springer, 2020, pp. 357–367, doi: 10.1007/978-3-030-41505-1_34.

[11] J. Mueller and A. Thyagarajan, "Siamese recurrent architectures for learning sentence similarity," in *Proc. AAAI Conf. Artif. Intell.*, Mar. 2016, vol. 30, no. 1, pp. 1–7, doi: 10.1609/aaai.v30i1.10350.

[12] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," 2019, *arXiv:1908.10084*.

[13] D. Cer, Y. Yang, S.-Y. Kong, N. Hua, N. Limtiaco, R. S. John, N. Constant, M. Guajardo-Cespedes, S. Yuan, C. Tar, Y.-H. Sung, B. Strope, and R. Kurzweil, "Universal sentence encoder," 2018, *arXiv.1803.11175*.

[14] J. Bromley, I. Guyon, Y. LeCun, E. Säckinger, and R. Shah, "Signature verification using a 'Siamese' time delay neural network," in *Proc. Adv. Neural Inf. Process. Syst.* San Mateo, CA, USA: Morgan Kaufmann, 1993, pp. 1–8. Accessed: May 24, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/1993/hash/288cc0ff022877bd3df94bc9360b9c5d-Abstract.html

[15] F. Rabbi and M. S. Siddik, "Detecting code comment inconsistency using Siamese recurrent network," in *Proc. 28th Int. Conf. Program Comprehension (ICPC)*. New York, NY, USA: ACM, Sep. 2020, pp. 371–375, doi: 10.1145/3387904.3389286.

[16] D. Chicco, "Siamese neural networks: An overview," in *Artificial Neural Networks*, H. Cartwright, Ed., New York, NY, USA: Springer, 2021, pp. 73–94, doi: 10.1007/978-1-0716-0826-5_3.

[17] E. L. Pontes, S. Huet, A. C. Linhares, and J.-M. Torres-Moreno, "Predicting the semantic textual similarity with Siamese CNN and LSTM," 2018, *arXiv:1810.10641*.

[18] S. Chopra, R. Hadsell, and Y. LeCun, "Learning a similarity metric discriminatively, with application to face verification," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. (CVPR)*, vol. 1, Jun. 2005, pp. 539–546, doi: 10.1109/CVPR.2005.202.

[19] G. R. Koch. (2015). *Siamese Neural Networks for One-Shot Image Recognition*. Accessed: May 24, 2024. [Online]. Available: https://www.semanticscholar.org/paper/Siamese-Neural-Networks-for-One-Shot-Image-Koch/f216444d4f2959b4520c61d20003fa30a199670a

[20] L. Zhao, W. Alhoshan, A. Ferrari, K. J. Letsholo, M. A. Ajagbe, E.-V. Chioasca, and R. T. Batista-Navarro, "Natural language processing for requirements engineering: A systematic mapping study," *ACM Comput. Surv.*, vol. 54, no. 3, pp. 1–41, Apr. 2021, doi: 10.1145/3444689.

[21] *Zero-Shot Learning for Requirements Classification: An Exploratory Study—ScienceDirect*. Accessed: May 24, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950584923000563

[22] D. Bajaj, A. Goel, S. C. Gupta, and H. Batra, "MUCE: A multilingual use case model extractor using GPT-3," *Int. J. Inf. Technol.*, vol. 14, no. 3, pp. 1543–1554, May 2022, doi: 10.1007/s41870-022-00884-2.

[23] S. Reddivari and J. Wolbert, "Calculating requirements similarity using word embeddings," in *Proc. IEEE 46th Annu. Comput., Softw., Appl. Conf. (COMPSAC)*, Jun. 2022, pp. 438–439, doi: 10.1109/COMPSAC54236.2022.00079.

[24] W. Alhoshan, R. Batista-Navarro, and L. Zhao, "Semantic frame embeddings for detecting relations between software requirements," in *Proc. 13th Int. Conf. Comput. Semantics-Student Papers*, S. Dobnik, S. Chatzikyriakidis, V. Demberg, K. A. Kwaik, and V. Maraev, Eds., Gothenburg, Sweden: ACM, May 2019, pp. 44–51, doi: 10.18653/v1/w19-0606.

[25] *Densely Connected Bidirectional LSTM With Applications to Sentence Classification | SpringerLink*. Accessed: Jun. 26, 2024. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-319-99501-4_24

[26] J. Zhou, Y. Lu, H.-N. Dai, H. Wang, and H. Xiao, "Sentiment analysis of Chinese microblog based on stacked bidirectional LSTM," *IEEE Access*, vol. 7, pp. 38856–38866, 2019, doi: 10.1109/ACCESS.2019.2905048.

[27] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.* Curran Associates, 2017, pp. 1–11. Accessed: May 25, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[28] *PROMISE Software Engineering Repository*. Accessed: May 24, 2024. [Online]. Available: http://promise.site.uottawa.ca/SERepository/

[29] A. Ferrari, G. O. Spagnolo, and S. Gnesi, "PURE: A dataset of public requirements documents," in *Proc. IEEE 25th Int. Requirements Eng. Conf. (RE)*, Sep. 2017, pp. 502–505, doi: 10.1109/RE.2017.29.

[30] *Using Semantic Frames for Measuring and Identifying Semantic Relationships in Software Descriptions—ProQuest*. Accessed: May 24, 2024. [Online]. Available: https://www.proquest.com/openview/d05d49c9890b689b4403991c34ac12e9/1?pq-origsite=gscholar&cbl=2026366&diss=y

[31] F. Ahmad and M. Faisal, "Assessing similarity between software requirements: A semantic approach," *Int. J. Inf. Eng. Electron. Bus.*, vol. 15, no. 2, pp. 38–53, Apr. 2023.

[32] *On the Relationship Between Similar Requirements and Similar Software | Requirements Engineering*. Accessed: May 25, 2024. [Online]. Available: https://link.springer.com/article/10.1007/s00766-021-00370-4

[33] F. A. Mihany, H. Moussa, A. Kamel, and E. Ezat, "A framework for measuring similarity between requirements documents," in *Proc. 10th Int. Conf. Informat. Syst. (INFOS)*. New York, NY, USA: ACM, May 2016, pp. 334–335, doi: 10.1145/2908446.2908510.

[34] F. A. Mihany, H. Moussa, A. Kamel, E. Ezzat, and M. Ilyas, "An automated system for measuring similarity between software requirements," in *Proc. 2nd Afr. Middle East Conf. Softw. Eng. (AMECSE)*. New York, NY, USA: ACM, May 2016, pp. 46–51, doi: 10.1145/2944165.2944173.

[35] A. Azzam, Y. Priyadi, and J. H. Husen, "Similarity software requirement specification (SRS) elicitation based on the requirement statement using text mining on the MNC play inventory management application," in *Proc. 4th Int. Conf. Comput. Inform. Eng. (IC2IE)*, Sep. 2021, pp. 123–128, doi: 10.1109/IC2IE53219.2021.9649023.

[36] V. Bertram, M. Boß, E. Kusmenko, I. H. Nachmann, B. Rumpe, D. Trotta, and L. Wachtmeister, "Neural language models and few shot learning for systematic requirements processing in MDSE," in *Proc. 15th ACM SIGPLAN Int. Conf. Softw. Lang. Eng. (SLE)*. New York, NY, USA: ACM, Dec. 2022, pp. 260–265, doi: 10.1145/3567512.3567534.

[37] V. Bertram, M. Boß, E. Kusmenko, I. H. Nachmann, B. Rumpe, D. Trotta, and L. Wachtmeister, "Technical report on neural language models and few-shot learning for systematic requirements processing in MDSE," 2022, *arXiv:2211.09084*.

[38] G. Malik, M. Cevik, S. Bera, S. Yildirim, D. Parikh, and A. Basar, "Software requirement-specific entity extraction using transformer models," in *Proc. Can. Conf. Artif. Intell.*, May 2022, pp. 1–12, doi: 10.21428/594757db.9e433d7c.

[39] L. Shi, M. Xing, M. Li, Y. Wang, S. Li, and Q. Wang, "Detection of hidden feature requests from massive chat messages via deep Siamese network," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng. (ICSE)*. New York, NY, USA: ACM, Oct. 2020, pp. 641–653, doi: 10.1145/3377811.3380356.

[40] G. Malik, M. Cevik, D. Parikh, and A. Basar, "Supervised semantic similarity-based conflict detection algorithm: S3CDA," 2022, *arXiv:2206.13690*.

[41] G. Malik, S. Yildirim, M. Cevik, A. Bener, and D. Parikh, "Transfer learning for conflict and duplicate detection in software requirement pairs," 2023, *arXiv:2301.03709*.

[42] A. Nayak, H. P. Timmapathini, V. Murali, and A. A. Gohad, "Few-shot learning approaches for classifying low resource domain specific software requirements," 2023, *arXiv:2302.06951*.

[43] T. Hey, J. Keim, A. Koziolek, and W. F. Tichy, "NoRBERT: Transfer learning for requirements classification," in *Proc. IEEE 28th Int. Requirements Eng. Conf. (RE)*, Aug. 2020, pp. 169–179, doi: 10.1109/RE48521.2020.00028.

[44] J. Mullis, C. Chen, B. Morkos, and S. Ferguson, "Deep neural networks in natural language processing for classifying requirements by origin and functionality: An application of BERT in system requirements," *J. Mech. Des.*, vol. 146, pp. 1–14, Nov. 2023, Art. no. 041401, doi: 10.1115/1.4063764.

[45] S. Das, N. Deb, A. Cortesi, and N. Chaki, "Sentence embedding models for similarity detection of software requirements," *Social Netw. Comput. Sci.*, vol. 2, no. 2, p. 69, Feb. 2021, doi: 10.1007/s42979-020-00427-1.

[46] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.

[47] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.

[48] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.* Glasgow, U.K.: Curran Associates, 2020, pp. 1877–1901. Accessed: May 25, 2024. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html

[49] J. Achiam et al., "GPT-4 technical report," 2024, *arXiv.2303.08774*.

[50] Z. Yang, Z. Dai, Y. Yang, J. Carbonell, R. R. Salakhutdinov, and Q. V. Le, "XLNet: Generalized autoregressive pretraining for language understanding," in *Proc. Adv. Neural Inf. Process. Syst.* Curran Associates, 2019, pp. 1–18. Accessed: May 25, 2024. [Online]. Available: https://proceedings.neurips.cc/paper/2019/hash/dc6a7e655d7e5840e66733e9ee67cc69-Abstract.html

[51] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Proc. Adv. Neural Inf. Process. Syst.* Glasgow, U.K.: Curran Associates, 2020, pp. 5776–5788. Accessed: May 25, 2024. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[52] J. Ni, G. H. Ábrego, N. Constant, J. Ma, K. B. Hall, D. Cer, and Y. Yang, "Sentence-T5: Scalable sentence encoders from pre-trained text-to-text models," 2021, *arXiv:2108.08877*.

[53] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.

[54] W. Alhoshan, L. Zhao, A. Ferrari, and K. J. Letsholo, "A zero-shot learning approach to classifying requirements: A preliminary study," in *Requirements Engineering: Foundation for Software Quality*, V. Gervasi and A. Vogelsang, Eds., Cham, Switzerland: Springer, 2022, pp. 52–59, doi: 10.1007/978-3-030-98464-9_5.

[55] R. K. Helmeczi, M. Cevik, and S. Yıldırım, "Few-shot learning for sentence pair classification and its applications in software engineering," 2023, *arXiv:2306.08058*.

[56] D. Chandrasekaran and V. Mago, "Evolution of semantic similarity—A survey," *ACM Comput. Surv.*, vol. 54, no. 2, pp. 1–37, Feb. 2021, doi: 10.1145/3440755.

[57] P. Riedel, M. Reichert, R. Von Schwerin, A. Hafner, D. Schaudt, and G. Singh, "Performance analysis of federated learning algorithms for multilingual protest news detection using pre-trained DistilBERT and BERT," *IEEE Access*, vol. 11, pp. 134009–134022, 2023, doi: 10.1109/ACCESS.2023.3334910.

[58] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," 2020, *arXiv:2001.08361*.

[59] K. S. Kalyan, A. Rajasekharan, and S. Sangeetha, "AMMUS: A survey of transformer-based pretrained models in natural language processing," 2021, *arXiv.2108.05542*.

[60] H. Luo. (2023). *Prompt-Learning and Zero-Shot Text Classification With Domain-Specific Textual Data*. Accessed: May 25, 2024. [Online]. Available: https://urn.kb.se/resolve?urn=urn:nbn:se:uu:diva-504646

[61] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Trans. Signal Process.*, vol. 45, no. 11, pp. 2673–2681, Nov. 1997, doi: 10.1109/78.650093.

[62] H. Gudaparthi, R. Johnson, H. Challa, and N. Niu, "Deep learning for smart sewer systems: Assessing nonfunctional requirements," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng., Softw. Eng. Soc. (ICSE-SEIS)*. New York, NY, USA: ACM, Oct. 2020, pp. 35–38.

[63] *LangChain: Building Applications With Large Language Models*. Accessed: May 28, 2024. [Online]. Available: https://python.langchain.com/v0.1/docs/get_started/introduction/

[64] G. Canbek, S. Sagiroglu, T. T. Temizel, and N. Baykal, "Binary classification performance measures/metrics: A comprehensive visualized roadmap to gain new insights," in *Proc. Int. Conf. Comput. Sci. Eng. (UBMK)*, Oct. 2017, pp. 821–826, doi: 10.1109/UBMK.2017.8093539.

[65] Q. Li, H. Peng, J. Li, C. Xia, R. Yang, L. Sun, P. S. Yu, and L. He, "A survey on text classification: From traditional to deep learning," *ACM Trans. Intell. Syst. Technol.*, vol. 13, no. 2, pp. 1–41, Apr. 2022, doi: 10.1145/3495162.

[66] H. K. Wright, M. Kim, and D. E. Perry, "Validity concerns in software engineering research," in *Proc. FSE/SDP Workshop Future Softw. Eng. Res.* Santa Fe, NM, USA: ACM, Nov. 2010, pp. 411–414, doi: 10.1145/1882362.1882446.

NOJOOM A. ALNAJEM (Member, IEEE) received the M.S. degree in software engineering from King Saud University, Riyadh, Saudi Arabia, where she is currently pursuing the Ph.D. degree with the Department of Software Engineering, College of Computer and Information Sciences. In recent years, she was involved with Saudi Arabia Section of the Instrumentation and Measurement Society Chapter. Her research interests include software engineering, RE, multimedia, AI, NLP for software engineering, and LLMs.

MANAL BINKHONAIN received the Ph.D. degree in computer science from the University of Manchester, in 2021. She is currently an Assistant Professor with the Department of Software Engineering, College of Computer and Information Sciences, King Saud University, Riyadh, Saudi Arabia. She is also the Chair of the Department of Software Engineering. Her current research interest includes the intersection of NLP with software engineering.

M. SHAMIM HOSSAIN (Senior Member, IEEE) received the Ph.D. degree in electrical and computer engineering from the University of Ottawa, ON, Canada, in 2009. He is currently a Professor with the Department of Software Engineering, College of Computer and Information Sciences, King Saud University, Riyadh, Saudi Arabia. He is also an Adjunct Professor with the School of Electrical Engineering and Computer Science, University of Ottawa. He has authored or co-authored more than 400 publications, including refereed journals (more than 280 SCI/ISI-indexed papers, more than 150 IEEE/ACM Transactions/journal articles, more than 23 ESI highly cited papers, two hot papers, conference papers, books, and book chapters. Recently, he co-edited a book *Connected Health in Smart Cities* (Springer). His research interests include cloud networking, smart environment (smart city and smart health), AI, DL, edge computing, the Internet of Things (IoT), multimedia for health care, and multimedia big data. He was a recipient of several awards, including the Best Conference Paper Award, the 2024 IEEE Communications Society Outstanding Paper Award, the 2016 ACM Transactions on Multimedia Computing, Communications, and Applications (TOMM) Nicolas D. Georganas Best Paper Award, the 2019 King Saud University Scientific Excellence Award (Research Quality), and the Research in Excellence Award from the College of Computer and Information Sciences (CCIS), King Saud University (3 Times in a Row). He has served as the co-chair, the general chair, the workshop chair, the publication chair, and a TPC at several IEEE and ACM conferences. He is also the Chair of the IEEE Special Interest Group on Artificial Intelligence (AI) for Health with the IEEE ComSoc EHealth Technical Committee. He is also the Symposium Chair of Selected Areas in Communications (E-Health) with IEEE GLOBECOM 2024. He is also the Track Chair of the IEEE International Conference on Consumer Electronics (ICCE 2024). He is the Technical Program Co-Chair of ACM Multimedia 2023. Currently, he is the Chair of Saudi Arabia Section of the Instrumentation and Measurement Society Chapter. He is on the editorial board of IEEE Transactions on Instrumentation and Measurement (IEEE TIM), IEEE Transactions on Multimedia (IEEE TMM), IEEE Transactions on Consumer Electronics (TCE), *ACM Transactions on Multimedia Computing, Communications, and Applications* (TOMM), IEEE Multimedia, *IEEE Network*, IEEE Wireless Communications, *Journal of Network and Computer Applications* (Elsevier), *International Journal of Multimedia Tools and Applications* (Springer), and *Games for Health Journal*. He has served as a Lead Guest Editor for more than two dozen special issues (SIs), including *ACM Transactions on Multimedia Computing, Communications, and Applications* (TOMM), *ACM Transactions on Internet Technology*, IEEE Transactions on Consumer Electronics, *IEEE Communications Magazine*, *IEEE Network*, IEEE Transactions on Information Technology in Biomedicine (currently JBHI), and IEEE Transactions on Cloud Computing. He is a Distinguished Member of the ACM. He is an IEEE Distinguished Lecturer (DL). He is a Highly Cited Researcher in Computer Science (Web of Science).